

BABAPPA: a simulation-trained, alignment-ensemble, site-level framework for branch-site selection inference under saturation and alignment uncertainty

Krishnendu Sinha

Department of Zoology, Jhargram Raj College, Jhargram, West Bengal, India

Correspondence: dr.krishnendusinha@gmail.com

Abstract

Inference of adaptive molecular evolution from coding sequences remains strongly dependent on multiple sequence alignment (MSA). However, alignment uncertainty, recombination, substitutional saturation, synonymous-rate variation, multinucleotide substitutions, insertion-deletion processes, and related forms of model misspecification can distort downstream selection analyses. Here I present the full methodological specification of BABAPPA (**B**ranched-site **A**lignment-**B**ias-**A**ware **P**robabilistic **P**ositive-selection **A**nalyzer), a simulation-trained framework for branch-site selection inference that is robust to alignment-induced artifacts and explicitly aware of saturation-driven information loss. The method is not optimized for any single alignment or any single codon model.

BABAPPA is built around six principles: large-scale simulator-truth training data with tracked ancestral histories; alignment ensembles generated with multiple codon-aware workflows; supervised multi-task learning that separates alignment-invariant signal from aligner-specific artifact; explicit saturation modeling from latent multiple-hit burden and observed proxies; calibration across null, hard-negative, and out-of-distribution regimes; and benchmarking against likelihood-based and alignment-uncertainty-aware baselines.

The framework also includes an auxiliary second-stage module for alignment-integrated ancestral sequence reconstruction (ASR) and protein-energy interpretation. This auxiliary layer is not primary evidence for positive selection. Instead, it provides mechanistic context for high-confidence BABAPPA hits by propagating alignment uncertainty into ancestral-state ensembles and quantifying how candidate substitutions alter predicted structural or energetic constraints.

This article is intentionally self-contained and written as an executable methods blueprint. It specifies the generative simulator, tensorization scheme, neural architecture, loss functions, calibration strategy, evaluation design, software modules, auxiliary ASR/energy components, expected file formats, and reporting schema. It also records the current engineering evidence so that future empirical claims are not confused with simulation-supervised validation.

The accompanying implementation has reached version 0.5.1-alpha. Development has now moved beyond direct whole-gene classification, which proved unstable under four-tier saturation stress tests, to an explicit branch-site truth workflow with conservative feature policy. A completed 100,000-family Apple Silicon/MPS validation used 25,000 simulated families per saturation tier, 8 taxa, 300 codons, and the production-fast alignment ensemble `identity`, `MAFFT`, `BABAPPAalign`, and `MUSCLE`. All 104 tier-stage markers completed, the cross-tier truth audit reported `explicit_truth_ok` for all tiers, and no proxy foreground-taxon labels were detected. Branch-site neural test AUROC values were 0.9986, 0.9970,

0.9945, and 0.9900 for low, moderate, high, and extreme saturation tiers, respectively. Branch-level aggregation all-split AUROC values were 0.9998, 0.9994, 0.9988, and 0.9979, while branch-to-gene aggregation all-split AUROC values were 0.9997, 0.9996, 0.9995, and 0.9994. Destructive shuffled-branch-label controls remained near random, but context-only ablation still raised a shortcut warning. These are strong simulation-oracle results, not empirical biological claims. The implementation now also includes a deployable simulation-trained conservative-branch-site package, empirical input quality control, out-of-distribution applicability gates, pilot-panel tooling, and a first guarded WRKY diagnostic pilot. Final empirical positive-selection claims remain blocked until simulation-matched calibration and external reference interpretation are complete.

1 Introduction

Comparative analyses of protein-coding sequences often infer natural selection through codon models in which the ratio of nonsynonymous to synonymous substitution rates, $\omega = d_N/d_S$, serves as a central summary of selective pressure [14, 28, 39]. Branch, site, and branch-site models have made it possible to test hypotheses about lineage-specific and site-specific adaptive evolution, and the improved branch-site likelihood method [40] in particular became a standard approach for detecting episodic positive selection. Random-effects extensions such as BS-REL, aBSREL, MEME, and BUSTED further broadened the inferential scope by allowing selection to vary across sites and lineages [17, 26, 27, 34]. These methods are powerful and widely used, but they operate on a prerequisite that is typically treated as fixed: a single inferred alignment of homologous codons.

That prerequisite is not benign. Alignment errors can materially inflate false positive rates in positive-selection scans, especially when evolutionary divergence and insertion–deletion activity increase [15, 30]. Recombination can violate the assumption that one phylogeny describes the history of all sites, sometimes causing severe false positive behavior when ignored [3, 32]. Synonymous rate variation can yield large excesses of false positives if not modeled [7, 37]. Multinucleotide substitutions and related non-canonical mutational processes can also mislead classical codon-model selection inference [10, 23, 35]. Substitutional saturation is a distinct confounder that must be handled explicitly: hidden multiple hits can erase historical signal, compress observed synonymous divergence, and drive branch-site tests toward low power or unstable interpretation, particularly for deeper divergences [5, 9, 13, 38]. Recent work such as BUSTED-E demonstrates that even heavily curated genome-scale alignments may retain residual local errors severe enough to distort inference, motivating explicit modeling of artifact-like regimes [33]. At the same time, the first deep-learning selection studies have shown that simulation-trained neural models can, in principle, learn useful signals from aligned sequence data [36]. The field is therefore ready for a new class of methods that does not discard codon-model reasoning, but no longer treats a single MSA as unquestioned truth and no longer treats highly saturated sequence space as equally informative as lightly diverged sequence space.

The objective of BABAPPA is to address this problem directly. The framework is designed for robust branch-site selection inference under alignment uncertainty by training on large ensembles of simulated orthogroups for which the full latent evolutionary history is known. Rather than asking a model to imitate the output of CODEML or any other existing pro-

gram, BABAPPA uses simulator-truth labels and explicitly exposes the learner to confounding processes that often generate false or distorted selection signals in practice. Multiple aligners are run on the same simulated family, creating several imperfect homology hypotheses for one underlying evolutionary history. The model is then trained to identify which signal patterns are consistent with true branch-site selection and which are specific to one aligner, one artifact class, or one misspecified regime.

This paper is written as a complete methods specification rather than as a minimal software note. Its purpose is twofold. First, it defines the scientific and statistical target of BABAPPA with enough precision that later performance claims can be judged against an explicit design. Second, it is intended to be implementation-ready: a new research assistant, developer, or coding agent should be able to use this document alone to build the simulator, data pipeline, training stack, benchmarking scripts, and reporting workflow. Where information is not yet available because the downstream execution has not yet been run, explicit placeholders are provided instead of silent omissions.

A second conceptual point is equally important. Ancestral reconstruction and biophysical landscape analysis are biologically informative, but they do not by themselves constitute a primary statistical test of positive selection. Recent work shows that integrating alignment uncertainty can improve ASR accuracy and downstream structural inference, and that modern ASR formulations can incorporate richer sequence models and epistatic information [1, 6, 8, 25]. For that reason, the present manuscript incorporates ASR and protein-energy analysis only as an auxiliary interpretation layer that operates after robust branch-site candidates have been identified. This preserves a clear separation between *selection inference* and *mechanistic interpretation*, which I argue is the most defensible way to unite the two directions within a single methods paper.

2 Conceptual framing: simulator-truth, observational truth, and identifiability

A central problem in this area is misuse of the phrase “ground truth.” In real comparative genomic datasets, the true historical locations of adaptive events are generally unknown. Accordingly, BABAPPA distinguishes three inferential layers.

Simulator-truth refers to the latent evolutionary history used to generate synthetic orthogroups: the true branch tree or local segment trees, true codon homology map, true mutation events, true insertion and deletion events, true recombination breakpoints, and the true subset of branch-site pairs assigned to adaptive regimes. Only simulation provides this level of truth.

Analysis truth refers to the output of a downstream method applied to a particular representation of the data, such as a single alignment analyzed by CODEML, MEME, or aBSREL. Analysis truth is not historical truth; it is method-specific inference.

Observational identifiability refers to whether the available extant sequences contain sufficient information to recover a meaningful approximation to the simulator-truth signal. Some branch-site events are intrinsically difficult to recover once indels, short branches, synonymous-site saturation, codon-position-specific multiple hits, or recombination obscure site homology

and event localization. In BABAPPA, severe saturation is not merely another nuisance covariate; it is treated as a mechanism of information destruction that can render a confident branch-site call scientifically indefensible even when a classifier remains numerically decisive.

For this reason, BABAPPA is framed as a *robust branch-site inference system* rather than an oracle that can always recover exact historical selection. The framework is trained to predict simulator-truth labels, but it also learns an *artifact probability* and an *out-of-distribution or low-identifiability probability*. This distinction is essential. A method that forces every ambiguous case into a positive-selection call may achieve attractive sensitivity in some simulation slices while being scientifically unusable on real data.

3 Overview of the BABAPPA framework

BABAPPA is organized as a staged inference pipeline from simulation to deployable reporting. Rather than treating these as independent checklist items, the stages are designed as one information flow in which simulator-truth supervision, alignment-ensemble uncertainty, calibration, and operating-point selection are coupled. Table 1 summarizes the computational stages and their scientific role.

Table 1: High-level computational stages in the BABAPPA framework.

Stage	Role in the end-to-end inference flow
Orthogroup simulation and truth extraction	Generate coding-sequence families under controlled nuisance processes and record branch-, site-, segment-, and artifact-level simulator truth.
Alignment-ensemble generation	Produce alternative codon-aware alignments per family so that inference is trained against homology uncertainty rather than a single MSA hypothesis.
Tensorization and feature construction	Convert alignments, trees, and metadata into trainable tensor objects with explicit channels for sequence, gap, and later saturation or artifact covariates.
Neural inference	Learn multi-task outputs for adaptive signal, artifact likelihood, and reliability-related quantities using branch- and gene-level supervision.
Calibration and threshold policy	Fit probability calibration and derive empirical operating points (for example conservative FDR control versus balanced detection).
Benchmarking and stress testing	Compare against codon-model baselines and evaluate robustness under saturation, indels, recombination, and other hard-negative regimes.
Empirical deployment and reporting	Apply frozen models to real orthogroups and export calibrated machine-readable and manuscript-ready outputs with uncertainty context.

The key novelty lies not in deep learning per se, but in the use of *alignment ensembles with*

known latent history to separate stable evolutionary signal from aligner-specific distortion while explicitly learning when saturation has pushed the data beyond a regime in which branch-site localization remains identifiable.

4 Current implementation status

The present manuscript remains a full methods blueprint for the final branch-site BABAPPA system, but the accompanying software has advanced substantially since the first gene-level substrate. The current development release is 0.5.1-alpha. It is important to distinguish the implemented system from the final architecture described later in this manuscript. Version 0.5.1-alpha is still simulation-supervised and oracle-labeled; it does *not* yet constitute empirical branch-site inference, and it does not yet include final artifact, saturation-risk, abstention, ASR, or protein-energy heads. However, it now implements the critical explicit branch-site truth layer needed to bridge simulator truth to future branch-site inference.

The implemented components are summarized in Table 2. Collectively, these establish a reproducible branch-site research substrate with validated four-tier saturation performance, branch-site neural scoring, branch-level and branch-to-gene aggregation, null/decoy controls, and completed explicit branch-truth validation at 100,000 simulated families on Apple Silicon/MPS.

Table 2: Implemented capabilities in release 0.5.1-alpha (current executable substrate).

Implemented layer	Current status and outputs
Simulation and audit	CDS-family simulation with per-family FASTA/tree/truth/homology/events/metadata plus completeness, divergence, leakage, and saturation-tier audits.
Alignment and site maps	Internal <code>identity</code> scaffolds, quarantined <code>codon_dropout</code> noise controls, production-fast external MAFFT/BABAPPAalign/MUSCLE wrappers with automatic method quarantine, diagnostic-only PRANK and T-Coffee support, and aligned-site to simulated-site map validation.
Tensor and dataset stack	NumPy tensor shards, manifests, audits, feature extraction, deterministic family-disjoint split/index management, and multi-source saturation dataset merging.
Site-label and site-dataset stack	Oracle selected-site extraction, mapped aligned-site labeling, site-level feature tables, leakage audits, mappable-site filtering, and downsampling for severe class imbalance.
Explicit branch-site truth stack	Direct simulator branch-site truth extraction, conservative branch-site feature policy, streamed and capped branch-site datasets, truth-status audits, and proxy-label rejection.
Site-level learning	NumPy logistic site baseline and optional-PyTorch site neural classifier with train-only standardization, class weighting, validators, and stratified metrics.

Calibration, thresholds, and aggregation	Temperature and quantile site calibration, site and aggregation threshold policies, site-to-gene aggregation, aggregation null/decoy controls, and calibration comparison.
Evaluation and reporting	Stratified evaluation by split/tier/method, site model comparison, repeated-seed stability benchmarks, report generation, run summaries, and large-run planning.

This implementation sequence changes the manuscript status in two ways. First, the project is no longer only a conceptual proposal: the repository now contains a reproducible end-to-end command-line workflow from simulation through site-label extraction, site-dataset construction, site baseline and neural training, calibration, threshold profiling, stratified evaluation, site-to-gene aggregation, null controls, external-aligner method policy, and reporting. Second, all current performance claims must remain explicitly simulation-oracle and site-level. Branch-conditioned empirical inference, cross-aligner consensus heads, saturation-risk heads, artifact heads, abstention heads, and the auxiliary ASR/energy layer remain planned methodological targets rather than completed biological results.

5 Generative simulator for orthogroup evolution

5.1 General requirements

The simulator is the scientific core of BABAPPA. If its generative assumptions are narrow, the trained model will merely memorize one stylized world. Therefore the simulator must support broad, randomized, and explicitly parameterized variation across tree shapes, sequence lengths, codon compositions, background selection regimes, indel processes, recombination intensity, synonymous-rate variation, multinucleotide mutations, and branch-site adaptive events.

Each synthetic family g is represented by a tuple

$$\mathcal{D}_g = (T_g, \mathcal{A}_g, \mathcal{H}_g, \mathcal{Y}_g, \Theta_g),$$

where T_g is the species tree or segment-wise local trees, $\mathcal{A}_g$ is the ancestral history, $\mathcal{H}_g$ is the tracked codon homology map, $\mathcal{Y}_g$ is the set of truth labels, and Θ_g is the vector of generative parameters.

5.2 Tree generation

For each family, the simulator draws a rooted phylogeny $T_g = (V_g, E_g)$ with n_g taxa. The simplest default is a Yule pure-birth tree, but the framework should support parametric trees (Yule or birth–death with sampled branch lengths), template trees rescaled from empirical orthogroups, and hybrid trees that combine empirical topologies with randomized branch-length perturbations.

The number of taxa n_g should be sampled over a broad range, for example

$$n_g \sim \text{DiscreteUniform}(8, 64),$$

unless an experiment fixes the value. Branch lengths should be drawn under a mixture distribution so that shallow, moderate, and deep divergences are all represented. A practical default is to sample a mean tree depth and then rescale branches with branch-specific multipliers:

$$\ell_b = \tau_g \cdot \rho_b,$$

where τ_g controls family-level divergence and ρ_b is a branch-specific rate multiplier.

5.3 Root sequence generation

Each family begins with a root coding sequence of length L_g codons:

$$L_g \sim \text{DiscreteUniform}(100, 2000),$$

or from an empirical length distribution if desired. Root codons can be generated either by a parametric root procedure (codons sampled from stationary frequencies $\boldsymbol{\pi}_g$) or by a template-root procedure (codons sampled from empirical coding regions while preserving amino-acid composition and codon bias).

To encourage realism, codon frequencies should be family-specific:

$$\boldsymbol{\pi}_g \sim \text{Dirichlet}(\boldsymbol{\alpha}_\pi),$$

with optional GC and codon-usage constraints.

Each root codon is assigned an immutable latent homology identifier. When insertion events occur later, newly inserted codons receive new identifiers together with a birth branch and birth time. This explicit homology bookkeeping is crucial because it makes training labels unambiguous even when downstream alignments disagree.

5.4 Background codon evolution

The background substitution process can be expressed as an MG94- or GY94-style codon model [14, 28]. For branch b and site i , define an instantaneous rate matrix

$$Q_{b,i} = \rho_b \alpha_i \sigma_i Q_{\text{codon}}(\kappa_g, \boldsymbol{\pi}_g, \omega_{b,i}, \eta_g),$$

where ρ_b is the branch multiplier, α_i is the site-specific rate multiplier, σ_i is an optional synonymous-rate modifier, κ_g is the transition/transversion bias, $\boldsymbol{\pi}_g$ is the codon-frequency vector, $\omega_{b,i}$ is the branch-site nonsynonymous pressure, and η_g controls the probability of multi-nucleotide events when the generalized model is used.

The site-specific factors should be sampled from broad distributions, e.g.

$$\alpha_i \sim \text{Gamma}(k_\alpha, \theta_\alpha), \quad \sigma_i \sim \text{LogNormal}(\mu_\sigma, \tau_\sigma^2).$$

This allows pervasive rate heterogeneity and synonymous-rate variation, both of which are known to affect selection inference [7, 37].

5.5 Branch-site adaptive events

The core labels of interest are branch-site adaptive events. For each family, a subset of branches is assigned as event-eligible:

$$Z_{g,b}^{(\text{branch})} \in \{0, 1\}.$$

For eligible branches, a subset of latent homologous codon identifiers is selected as adaptive targets:

$$Z_{g,b,h}^{(\text{site})} \in \{0, 1\}, \quad h \in \mathcal{H}_g.$$

The adaptive regime can then be imposed by setting

$$\omega_{b,h} = \begin{cases} \omega_{\text{bg},h} & \text{if } Z_{g,b,h}^{(\text{site})} = 0, \\ \omega_{\text{sel},g,b,h} & \text{if } Z_{g,b,h}^{(\text{site})} = 1, \end{cases}$$

with $\omega_{\text{sel},g,b,h} > 1$ and $\omega_{\text{bg},h} \leq 1$ unless a relaxed-constraint regime is being simulated separately.

To avoid an unrealistically simplistic binary world, adaptive events should vary in strength (ω_{sel}), duration (entire branch versus branch segment), prevalence (fraction of sites affected), clustering (independent sites versus contiguous hotspots), and recurrence (single-branch versus repeated or convergent events).

The simulator must also include difficult near-null scenarios such as short foreground branches, weak positive selection, and very sparse selected sites.

5.6 Insertion and deletion processes

Indels are indispensable because alignment uncertainty is one of the main targets of the framework. The simulator must therefore record not only final sequences, but also the true history of insertion and deletion events and the resulting homology map. Indels can be generated under a length-dependent distribution as in INDELible [12], but the exact mechanism is configurable. A practical default is:

$$N_{\text{indel},b} \sim \text{Poisson}(\lambda_{\text{indel},g} \ell_b),$$

with lengths sampled from a geometric or Zipf-like distribution. Insertions and deletions should be parameterized separately because asymmetric processes are common in real datasets.

5.7 Recombination and local genealogies

Because recombination can invalidate the single-tree assumption of codon-model inference [3, 32], BABAPPA includes recombination as an explicit nuisance process. The simulator should support two practical modes.

Mode 1: breakpoint mosaic simulation. The coding sequence is partitioned into contiguous segments, each segment using its own local tree obtained by nearest-neighbor-interchange or subtree-prune-regraft perturbation of the global tree.

Mode 2: ARG-inspired approximation. Breakpoints are sampled along the sequence and local genealogies are generated from a simplified ancestral recombination graph approximation.

Each family stores breakpoint locations and local trees. Importantly, recombination is *not* treated as positive selection, but as a potential confounder that may create false or fragmented branch-site signal in downstream analyses.

5.8 Substitutional saturation as an explicit confounder

Saturation cannot be handled adequately by simply allowing long branches in the simulator. In this context, saturation refers to the accumulation of repeated substitutions at the same historical positions such that the observed alignment no longer preserves a faithful record of the underlying number, type, and temporal order of changes [9, 38]. For branch-site inference, the most relevant consequence is not only general information loss, but specifically the erosion or compression of synonymous divergence, local codon-state history, and sitewise branch attribution [5, 13].

Accordingly, BABAPPA treats saturation at two distinct levels.

Latent saturation. Because the simulator tracks the full mutational history, it can compute branch-, segment-, and family-level multiple-hit burden directly from the generative record. A practical latent statistic is

$$S_{g,r}^{(\text{latent})} = \frac{\#\{\text{historical nucleotide changes in region } r\} - \#\{\text{observable terminal differences in } r\}}{\max(1, \#\{\text{historical nucleotide changes in region } r\})},$$

where region r may denote a family, segment, codon-position class, or branch-conditioned window. Values near zero indicate little hidden change, whereas values near one indicate strong masking of historical substitutions.

Observed saturation proxies. Real data do not provide latent histories, so the simulator must also emit observed diagnostics that a deployed method could compute. Recommended proxies include: Xia’s index of substitution saturation (Iss) or its codon-position-specific analogues [38]; entropy-based saturation scores [9]; transition/transversion flattening curves; pairwise uncorrected-versus-corrected distance residuals; branch-length compression relative to simulated truth; and position-specific estimates of synonymous divergence exhaustion.

Simulation of saturation should be decoupled from simple divergence depth by randomizing several contributing factors, including mean tree depth and branch-length heterogeneity, transition/transversion ratio, codon-position-specific rate multipliers, among-site rate variation, GC or compositional shifts interacting with multiple-hit processes, segment-specific acceleration under recombination, and foreground events embedded within otherwise deeply diverged backgrounds.

Families should then be stratified into at least four saturation tiers according to latent and observed summaries: low, moderate, high, and extreme. Extreme-saturation families are especially important because the scientifically appropriate behavior may be abstention rather than forced prediction. The objective of BABAPPA is therefore not to “correct” saturation away in all cases, but to learn the boundary between recoverable and nonrecoverable branch-site signal under saturation pressure.

5.9 Multinucleotide substitutions and model misspecification

Single-nucleotide codon transitions are insufficient for realism. Multinucleotide substitutions can create false signatures of selection when omitted from analysis [10, 23, 35]. Therefore, the simulator should include a tunable multinucleotide mechanism, for example by allowing instantaneous doublet or triplet changes with probability controlled by η_g .

Additional forms of model misspecification should also be simulated, including branch-specific GC shifts, heterogeneous synonymous substitution rates, branch-specific codon-bias changes, heterotachy, local compositional heterogeneity, and annotation-like artifacts such as frame disruptions, terminal sequence truncation, or erroneous short insertions.

The final class is especially important because BUSTED-E showed that residual local alignment or annotation errors can dominate apparent evidence for positive selection [33]. Hard-negative families with no adaptive events but strong artifact pressure should therefore be included deliberately.

5.10 Truth labels and metadata

For each family, the simulator writes a structured truth object with family labels (presence or absence of adaptive events), branch labels (branches containing one or more adaptive events), latent-homology labels (truly adaptive branch-homology pairs), artifact labels (deliberately injected artifact classes), nuisance labels (recombination, indel burden, synonymous-rate variation, multinucleotide burden, GC shifts, and related covariates together with latent and observed saturation summaries), and optional identifiability labels such as minimal divergence around adaptive sites, post-indel observability, or homology fragmentation.

The truth object is the reference against which all supervised training and benchmarking are defined.

6 Alignment ensemble generation

6.1 Rationale

A single simulated family should produce multiple competing alignments. Let $\mathcal{A} = \{a_1, \dots, a_K\}$ denote the set of aligners. In the production-fast implementation I recommend

$$\mathcal{A}_{\text{fast}} = \{\text{identity}, \text{BABAPPAlign}, \text{MAFFT}, \text{MUSCLE}\},$$

with version numbers recorded explicitly at runtime and with method-level quarantine before tensorization. MAFFT and MUSCLE provide fast, independent production aligner channels, while BABAPPAlign supplies a distinct protein-model-guided codon-alignment channel [11, 16]. PRANK, T-Coffee, PAGAN, and related phylogeny-aware or consistency-based workflows remain valuable diagnostic or held-out-aligner comparisons because they differ in gap treatment, phylogeny awareness, and alignment heuristics [21, 22, 29]. If additional aligners are added later, the framework should support held-out-aligner evaluation so that generalization to unseen alignment algorithms can be measured.

In the current executable release, external aligners are optional rather than mandatory dependencies. The command `check-aligners` detects internal `identity`, internal `codon_dropout`, MAFFT, PRANK, BABAPPAlign, MUSCLE, and T-Coffee backends when available. The command `align-external` can run a mixed internal/external ensemble, validate codon-frame compatibility, record failures, and quarantine methods whose family-level failure or site-map error rates exceed policy thresholds. `codon_dropout` remains useful as an unmappable noise-control channel, but it is quarantined from mapped oracle-label training because taxon-specific codon gaps can create consensus site-map conflicts. The default mapped external-aligner validation plan now uses `identity`, MAFFT, BABAPPAlign, and MUSCLE with automatic quarantine. PRANK remains diagnostic-only because it was slow and produced real high-tier frame errors during external validation, while T-Coffee remains optional diagnostic-only because of expected runtime cost.

6.2 Protein-guided and codon-aware workflows

For coding sequences, each aligner should be run in codon-aware fashion whenever possible. When an aligner is protein-based, the recommended workflow is translation of CDS to protein, protein alignment, back-translation to codon alignment, and codon-phase validation with stop-codon artifact removal. For aligners capable of direct codon alignment, the direct codon mode should also be tested.

Each family should therefore produce multiple alignment channels, including direct codon alignment, protein-guided codon alignment, an optional trimmed version, and an optional guide-tree-perturbed version. This induces a richer alignment ensemble than merely changing the aligner binary.

6.3 Cross-aligner homology projection during training

Because simulated data provide latent homology identifiers, each alignment column can be projected back to the latent homology map. Let c index a column produced by aligner a . Then

$$M_{g,a}(c) \subseteq \mathcal{H}_g$$

denotes the set of latent homology identifiers represented in that column. During training, columns that predominantly map to the same latent homology identifier can be tied by a consistency loss. This is the key device that allows a model to learn from conflicting alignments without pretending those alignments are identical.

For empirical data, latent homology identifiers are unknown. In deployment mode, site-level outputs are therefore reported conservatively: aligner-specific site scores are retained, and a cross-aligner consensus site score is reported only when correspondence is sufficiently stable. Branch-level and gene-level predictions remain the primary inferential outputs.

The implemented site-map layer operationalizes this requirement for simulation-supervised training. For every aligned codon column, `build-site-map` walks each taxon sequence against

the original simulated FASTA and records the mapping

$$\text{aligned_codon_site_index} \leftrightarrow \text{true_simulated_site_index}.$$

Each mapping row records whether the correspondence is unique, all-gap, conflicting, or frame-invalid, together with a mapping-confidence score. Site-label extraction can then operate in mapped mode: the supervised target y_{site} is attached to the aligned site only when the aligned column maps back to a trusted original simulated site. This prevents label corruption when real aligners insert gaps or shift codon columns.

7 Tensorization and input representation

7.1 Per-aligner tensor representation

For family g , aligner a , and alignment length $L_{g,a}$ over n_g taxa, construct a tensor

$$X_{g,a} \in \mathbb{R}^{n_g \times L_{g,a} \times C},$$

where channels C may include codon token embeddings, amino-acid token embeddings, gap indicators, codon degeneracy classes, per-column entropy and gap fraction, taxon-specific missing-data masks, optional alignment-confidence scores, guide-tree distance features, windowed saturation-proxy features (for example codon-position-specific entropy, transition/transversion flattening, or corrected-versus-uncorrected distance residuals), and branch-incidence masks for queried branches.

In addition to per-column channels, each family should provide global or segment-level covariates that summarize information loss beyond any single column. These may include latent saturation summaries during training, observed saturation proxies available at deployment time, pairwise divergence quantiles, codon-position-specific distance summaries, and window-level estimates of synonymous exhaustion. Such covariates can be concatenated to branch-query embeddings or supplied to late pooling layers so that the network can learn when apparently strong local evidence should be discounted because it arises in a globally saturated background.

The implemented tensor substrate is intentionally smaller than this final representation. Version 0.5.1-alpha writes one tensor shard per family and alignment method with shape $\text{taxa} \times \text{codons} \times \text{channels}$, where the current channels are codon ID and gap indicator. Separate JSON metadata and label files preserve family ID, method, gene-level label, selected-site summaries, foreground taxon where applicable, saturation tier, and mapped-site information when an external site map is supplied. The site-dataset layer then extracts one row per labeled codon site, including safe numeric site features, split/method/tier metadata, and the supervised target y_{site} . Truth-derived fields such as `selected_sites`, `n_selected_sites`, and `y_site` are explicitly excluded from predictive features by leakage audits and feature-selection rules.

The model should also receive a tree representation. A practical choice is a branch-incidence matrix or graph object:

$$G_g = (V_g, E_g, \Phi_g),$$

with node or edge attributes such as branch length, depth from root, descendant taxon count,

and optional local-tree flags in recombinant segments.

7.2 Branch-conditioned prediction

Since the inferential target is branch-site selection, the network must be conditioned on a queried branch. For each branch b , define a branch embedding vector

$$\mathbf{q}_{g,b} = f_{\text{branch}}(T_g, b),$$

where f_{branch} may encode branch length, topological depth, subtree size, sister-branch context, and whether the branch belongs to a recombinant segment.

The network then predicts:

$$p_{g,a,b,c}^{(\text{site})}, \quad p_{g,b}^{(\text{branch})}, \quad p_g^{(\text{gene})}, \quad p_{g,a,b,c}^{(\text{artifact})}, \quad \hat{s}_{g,b}^{(\text{risk})}, \quad p_g^{(\text{ood})}.$$

8 Neural architecture

8.1 Design goals

The architecture should satisfy five requirements: it must capture local column-wise and codon-wise patterns relevant to branch-site selection, incorporate explicit phylogenetic branch context, separate alignment-invariant signal from aligner-specific artifact, aggregate evidence across multiple alignments of the same family, and recognize saturation-driven information loss with support for abstention when branch-site localization becomes unreliable.

8.2 Proposed architecture

I recommend a four-module design.

Module I: per-aligner sequence encoder

Each alignment $X_{g,a}$ is passed through a per-aligner encoder E_θ shared across aligners:

$$H_{g,a} = E_\theta(X_{g,a}) \in \mathbb{R}^{L_{g,a} \times d}.$$

A practical implementation is a stack of 1D convolutional blocks followed by a lightweight Transformer or gated attention layers. Convolutions capture local codon and gap motifs; attention captures longer-range dependencies.

Module II: branch query conditioning

The branch embedding $\mathbf{q}_{g,b}$ is used in cross-attention against column embeddings:

$$U_{g,a,b} = \text{CrossAttn}(\mathbf{q}_{g,b}, H_{g,a}),$$

producing branch-conditioned column features. This makes the same alignment interpretable differently when querying different branches.

Module III: invariant versus artifact decomposition

To encourage the model to separate true biological signal from aligner-specific bias, each branch-conditioned embedding is decomposed into two latent streams:

$$U_{g,a,b} \rightarrow (S_{g,a,b}, R_{g,a,b}),$$

where S is intended to capture shared evolutionary signal and R captures aligner-specific residual structure. A gradient-reversal aligner-adversary is attached to S so that S becomes less predictive of aligner identity, while an artifact head is attached to R to explicitly model misalignment-like behavior. A parallel saturation-risk head receives pooled summaries from both streams together with global saturation covariates, because extreme saturation can destroy information even when aligner agreement is superficially high. This is not simply a regularization trick; it operationalizes the central scientific claim that robust branch-site signal should survive across multiple aligners more readily than artifacts do, but also that even consensus-like patterns should be down-weighted when the family resides in a saturation regime known to erase historical information.

Module IV: cross-aligner consensus aggregator

For a queried branch, each aligner yields a set of site logits and reliability scores. These are combined by a learned reliability-weighted pooling step:

$$\tilde{z}_{g,b,h} = \sum_{a=1}^K w_{g,a,b,h} z_{g,a,b,h}, \quad w_{g,a,b,h} = \frac{\exp(r_{g,a,b,h})}{\sum_{a'} \exp(r_{g,a',b,h})}.$$

Here h indexes latent homology identifiers during training; during deployment, h is replaced by conservative cross-aligner consensus sites when available.

Branch-level and gene-level probabilities are then produced by pooling over sites and branches:

$$p_{g,b}^{(\text{branch})} = 1 - \prod_h (1 - \sigma(\tilde{z}_{g,b,h})), \quad p_g^{(\text{gene})} = 1 - \prod_b (1 - p_{g,b}^{(\text{branch})}),$$

or by learned pooling layers if empirically superior.

The current neural implementation records an important design correction. Early direct gene-level models, including a contrastive neural-v2 classifier and later saturation-aware and ranking-aware variants, were useful diagnostic tools but were not stable on four-tier saturation benchmarks. Their probabilities frequently compressed near 0.5 and repeated-seed test AUROC remained close to random. Label-signal auditing then showed that the learnable signal was sparse and site-localized. The implemented production research layer therefore pivots to `SiteMLPClassifier`, a small optional-PyTorch multilayer perceptron trained on leakage-audited site features. Gene-level support is no longer learned directly from a whole alignment tensor; it is derived by aggregating site-level probabilities, with maximum site probability serving as the default gene

score. This is not yet the final branch-conditioned architecture, but it is the first implemented layer whose statistical target matches the biological sparsity of branch-site selection.

8.3 Interpretability outputs

The model should write interpretable auxiliary outputs for every prediction, including branch-specific saliency maps, per-aligner site scores, cross-aligner consensus site scores, artifact probability tracks, saturation diagnostic tracks with family-level saturation risk, uncertainty or out-of-distribution scores, and branch-level evidence decomposition by aligner.

These outputs are necessary if the method is to be used seriously by molecular evolution researchers rather than merely as a black-box classifier.

9 Training objective and calibration

9.1 Multi-task loss

The full loss function is

$$\mathcal{L} = \lambda_1 \mathcal{L}_{\text{site}} + \lambda_2 \mathcal{L}_{\text{branch}} + \lambda_3 \mathcal{L}_{\text{gene}} + \lambda_4 \mathcal{L}_{\text{artifact}} + \lambda_5 \mathcal{L}_{\text{consistency}} + \lambda_6 \mathcal{L}_{\text{saturation}} + \lambda_7 \mathcal{L}_{\text{calibration}} + \lambda_8 \mathcal{L}_{\text{adv}}.$$

The terms are defined as follows.

Site loss. Binary cross-entropy or focal loss on true branch–homology labels.

Branch loss. Binary cross-entropy on whether each branch contains at least one adaptive site.

Gene loss. Binary cross-entropy on whether a family contains any adaptive event.

Artifact loss. Binary cross-entropy for injected artifact classes and hard negatives.

Consistency loss. For columns across aligners that map to the same latent homology identifier, encourage similar site logits:

$$\mathcal{L}_{\text{consistency}} = \sum_{(a,c),(a',c') \mapsto h} \|z_{g,a,b,c} - z_{g,a',b,c'}\|_1.$$

Saturation loss. Regression or ordinal-classification loss for latent saturation burden and observed saturation tier. A practical implementation is to predict both a continuous latent saturation summary and a coarse class label (low/moderate/high/extreme), using mean-squared error plus cross-entropy. This auxiliary loss encourages the shared representation to encode information-destruction regimes rather than confusing them with positive selection.

Calibration loss. Brier score or temperature-scaled negative log-likelihood computed on a held-out calibration set.

Adversarial loss. A gradient-reversal aligner-classification loss to reduce aligner-specific information in the invariant latent stream.

9.2 Training data strata

The simulator should generate at least six training strata: clean null (no adaptive events with mild nuisance processes), clean positive (adaptive events without severe nuisance pressure), hard null (no adaptive events but strong confounders designed to fool classical methods), noisy positive (adaptive events mixed with indels, recombination, saturation, SRV, and MNMs), a saturation ladder spanning low/moderate/high/extreme tiers for matched null and positive families, and ambiguous or out-of-distribution regimes that are deliberately more extreme than the core training distribution.

This stratification is essential. If the model sees only clean positives and clean negatives, it cannot learn the artifact boundary that matters most.

9.3 Data splitting

Train, validation, calibration, and test splits must be family-disjoint. No family seed, local tree, or latent truth object may appear in more than one split. I recommend:

80% train, 10% validation, 5% calibration, 5% test,

with optional dedicated stress-test sets.

This splitting policy is implemented in `babappa index-dataset` and in the multi-source merger `babappa merge-datasets`. The default implementation assigns all methods from the same family to the same split, preventing leakage between alignment scaffolds of the same simulated history. Merged multi-saturation datasets use source-qualified family identifiers so that identically named families from different saturation-tier panels do not collide.

At least three special hold-outs should be created: a held-out aligner split in which one aligner or workflow is excluded from training, a held-out nuisance regime covering extreme SRV/recombination/indel burden or unseen saturation conditions, and a held-out tree regime with different taxon counts or tree shapes.

9.4 Probability calibration and decision thresholds

Raw neural probabilities are rarely perfectly calibrated. Therefore BABAPPA should use a dedicated calibration split to fit temperature scaling or isotonic regression for branch- and gene-level outputs. Site-level outputs may also be calibrated, but because site predictions are more variable, the default reporting should prioritize branch- and gene-level calibrated probabilities.

Decision thresholds should be derived empirically on the calibration split with explicit false-discovery control against null and hard-negative families. A practical deployment policy is to report a continuous calibrated probability, a conservative binary call only above a threshold chosen to control empirical false discovery on hard negatives, and an abstain flag when either $p^{(\text{ood})}$ exceeds a prespecified limit or saturation-risk signals indicate that information loss has crossed a preregistered boundary for reliable branch-site localization.

The current implementation supports calibration and operating-point selection at both site and aggregated gene levels. Gene-level threshold profiling remains available through `threshold-policy`,

while `calibrate-site-neural`, `site-threshold-policy`, and `aggregation-threshold-policy` implement the corresponding site-level and site-to-gene decision layers. Temperature scaling and a lightweight quantile calibration option are implemented for site neural predictions. In the 10,000-family site-level benchmark, quantile calibration substantially improved calibration error relative to temperature scaling, although strict site-level FDR remained difficult because individual selected sites are rare. This motivates reporting both continuous site probabilities and aggregation-level discovery profiles rather than treating a single site-level FDR threshold as the only scientifically relevant operating point.

10 Benchmarking and evaluation

10.1 Baselines

BABAPPA should be benchmarked against both classical and uncertainty-aware baselines, including the CODEML branch-site test [2, 39, 40], MEME and BUSTED [26, 27], aBSREL [34], BUSTED-E [33], and BALi-Phy-style alignment-integrated analyses on subsets where computation is feasible [30, 31].

Recombination-aware preprocessing using GARD may also be included where appropriate [19, 20]. Tree inference for alignment-based baselines may be performed with IQ-TREE 2 [24]. HyPhy 2.5 provides the baseline implementations for several episodic-selection tests [18].

10.2 Primary metrics

Evaluation must be reported at gene, branch, and site levels. Gene-level reporting should include AUROC, AUPRC, sensitivity, specificity, precision, Matthews correlation coefficient, Brier score or calibration error, and empirical false-discovery behavior on null and hard-negative sets. Branch-level reporting should include AUROC and AUPRC over branches, per-family branch ranking accuracy, top- k hit rates for true-event branches, and branch-probability calibration. Site-level reporting should include AUROC and AUPRC on simulated truth, localization overlap for selected sites, and consensus-restricted cross-aligner site accuracy.

10.3 Robustness metrics

Because robustness to alignment uncertainty is a central claim, additional metrics are needed.

Cross-aligner dispersion. For family g and branch b , define

$$\Delta_{g,b} = \text{Var}\left(p_{g,b}^{(\text{branch},a)} : a = 1, \dots, K\right).$$

Lower dispersion at fixed accuracy indicates greater stability.

Artifact discrimination. Measure the ability to separate true positives from hard negatives containing severe misalignment-like patterns but no adaptive events.

Held-out-aligner generalization. Evaluate performance when one aligner or one workflow is absent from training.

Saturation-specific evaluation. Because saturation is a primary confounder, every main metric should be stratified by saturation tier. At minimum, report AUROC, AUPRC, empirical false discovery, recall, and abstention rate separately for low, moderate, high, and extreme saturation. In addition, quantify whether the saturation-risk head is monotone with latent multiple-hit burden and with observed proxies such as Iss-like statistics or entropy saturation scores. A scientifically desirable pattern is not constant sensitivity at all costs, but controlled degradation with increasing abstention in the most saturated regimes.

This evaluation layer is now executable at gene and site levels. The command `stratified-eval` computes gene-level metrics by split, saturation tier, method, and their combinations. The site-specific command `site-stratified-eval` applies the same principle to site predictions, and `aggregate-sites` evaluates whether site evidence recovers gene-level simulator labels. Version 0.5.1-alpha also includes repeated-seed site-neural stability benchmarking, site baseline versus neural comparison, aggregation null/decoy controls, leakage audits, and large-run planning. The completed 10,000-family four-tier benchmark shows high site-level AUROC in every saturation tier, with graceful degradation from low to extreme saturation rather than the probability-collapse behavior observed in direct gene-level models.

Stress-test curves. Plot performance as functions of recombination rate, indel burden, saturation, SRV intensity, and MNM burden.

10.4 Statistical reporting

All benchmarking on test sets should report confidence intervals from repeated simulation batches or bootstrap resampling. Multiple-hypothesis situations, such as evaluating many families or many branches in empirical applications, should use false discovery rate control [4]. The manuscript-ready reporting layer should record both raw and adjusted P -value analogs where external likelihood baselines are used.

11 Software architecture and executable blueprint

11.1 Directory structure

I recommend the following repository structure.

```
BABAPPA/
|-- configs/
|   |-- simulator/
|   |-- aligners/
|   |-- training/
|   '-- benchmarks/
|-- babappa/
|   |-- simulate/
|   |-- align/
|   |-- tensors/
|   |-- models/
|   '-- training/
```

```

|   |-- calibration/
|   |-- benchmarks/
|   |-- asr/
|   |-- structure/
|   |-- energy/
|   |-- reports/
|   '-- utils/
|-- scripts/
|   |-- babappa_simulate.py
|   |-- babappa_align_ensemble.py
|   |-- babappa_build_tensors.py
|   |-- babappa_train.py
|   |-- babappa_calibrate.py
|   |-- babappa_predict.py
|   |-- babappa_benchmark.py
|   |-- babappa_asr.py
|   |-- babappa_energy.py
|   '-- babappa_make_report.py
|-- docs/
|-- tests/
'-- manuscript/

```

11.2 Minimal script responsibilities

`babappa_simulate.py`: generate family trees, root sequences, adaptive events, nuisance processes, final extant CDS, and truth metadata.

`babappa_align_ensemble.py`: run all configured aligners and workflows, produce codon alignments, and record runtime versions.

`babappa_build_tensors.py`: parse alignments, tree files, and truth metadata into tensor shards for efficient training.

`babappa_train.py`: train the multi-task network, log metrics, save checkpoints.

`babappa_saturation.py`: compute observed saturation diagnostics on simulated or empirical alignments and serialize the resulting family- and window-level covariates.

`babappa_calibrate.py`: fit temperature scaling or isotonic calibration on held-out calibration data.

`babappa_predict.py`: apply a frozen model to new orthogroups and produce family-, branch-, and site-level outputs.

`babappa_benchmark.py`: run baseline methods and aggregate comparative metrics.

`babappa_asr.py`: perform auxiliary alignment-integrated or alignment-ensemble ancestral sequence reconstruction for selected high-confidence families and branches, and propagate node-wise uncertainty into downstream summaries.

`babappa_energy.py`: score reconstructed ancestral and descendant protein states using configured structural or energetic engines and summarize branchwise changes in stability-like, interface-like, or conformational proxies.

`babappa_make_report.py`: generate TSV, JSON, plots, and manuscript-ready summaries.

11.3 Implemented command-line workflow in version 0.5.1-alpha

The repository currently exposes the workflow as a Typer command-line application rather than as separate scripts. The implemented commands are grouped as follows:

Command group	Implemented commands and outputs
Simulation and audit	<code>simulate</code> , <code>validate-sim</code> , and <code>audit-sim</code> generate CDS families and audit simulation completeness and divergence summaries.
Alignment and aligner status	<code>align-sim</code> , <code>align-external</code> , <code>check-aligners</code> , and <code>validate-align</code> generate internal and optional external alignments, record backend availability, and support method-specific timeout/failure handling.
Site-map projection and method policy	<code>build-site-map</code> , <code>validate-site-map</code> , <code>aligner-method-policy</code> , and <code>validate-aligner-method-policy</code> map aligned codon sites back to original simulated codon sites and quarantine methods with excessive failures, conflicts, or frame errors.
Tensorization	<code>build-tensors</code> and <code>validate-tensors</code> write NumPy tensor shards, tensor metadata, labels, manifests, and audit tables.
Dataset management	<code>index-dataset</code> , <code>validate-index</code> , <code>merge-datasets</code> , and <code>validate-merged-dataset</code> write feature tables, split tables, dataset indexes, and merged multi-source datasets.
Gene-level diagnostics	<code>train-baseline</code> , <code>train-neural</code> , <code>diagnose-neural</code> , <code>threshold-policy</code> , <code>stratified-eval</code> , and comparison validators remain available for historical gene-level diagnostics.
Site-label and site-dataset construction	<code>extract-site-labels</code> , <code>build-site-dataset</code> , <code>validate-site-dataset</code> , and <code>audit-site-leakage</code> implement oracle site-label extraction, mapped aligned-site labeling, feature extraction, and leakage checks.
Site modeling	<code>train-site-baseline</code> , <code>train-site-neural</code> , <code>validate-site-baseline</code> , and <code>validate-site-neural</code> implement NumPy and optional-PyTorch site-level models.
Site calibration and operating points	<code>calibrate-site-neural</code> , <code>site-threshold-policy</code> , <code>aggregation-threshold-policy</code> , and calibration-comparison validators implement site and aggregation decision policies.
Site aggregation and robustness	<code>site-stratified-eval</code> , <code>aggregate-sites</code> , <code>aggregation-controls</code> , <code>compare-site-models</code> , and <code>site-stability-benchmark</code> evaluate site scores, gene-level recovery, decoy controls, and repeated-seed stability.

Benchmark panels, planning, and summaries `make-saturation-panel,` `merge-datasets,`
 `plan-large-run,` `plan-external-aligner-validation,`
 `plan-fast-external-10k,` `summarize-external-tiers,`
 and their validators build tier-specific panels, generate
 user-run validation scripts without executing heavy jobs,
 and consolidate cross-tier reports.

The test suite is designed to keep these commands fast and reproducible. Neural tests are written to skip gracefully when PyTorch is unavailable, preserving a minimal dependency policy for non-neural users.

11.4 Recommended file schema

Each simulated family should write at least the following files:

File	Content
<code>family_id.fasta</code>	extant CDS sequences
<code>family_id.treefile</code>	global tree in Newick format
<code>family_id.local_trees.json</code>	optional local trees for recombinant segments
<code>family_id.truth.json</code>	branch-site labels, nuisance parameters, artifact labels
<code>family_id.homology.tsv</code>	mapping of extant codons to latent homology identifiers
<code>family_id.events.tsv</code>	all substitutions, indels, recombination breakpoints, MNMs
<code>family_id.meta.json</code>	parameter settings and provenance

Each alignment workflow should write:

File	Content
<code>family_id.{aligner}.codon.fasta</code>	codon alignment
<code>family_id.{aligner}.aa.fasta</code>	protein alignment if used
<code>family_id.{aligner}.map.tsv</code>	training-time projection of columns to latent homology IDs
<code>family_id.{aligner}.qc.json</code>	gap burden, entropy, length, confidence, runtime information
<code>family_id.{aligner}.asr.json</code>	optional per-aligner ancestral posterior summaries for selected nodes

11.5 Configuration philosophy

All experiments should be driven by versioned YAML or JSON configuration files, and no essential simulator or training choice should remain hard-coded. At minimum, each simulator configuration should declare family count, taxon-count distribution, sequence-length distribution, tree-generation mode, indel model and rate, recombination mode and rate, SRV distribution, MNM rate, adaptive-event frequency/strength/clustering, artifact-injection probabilities, and random-seed policy.

12 Auxiliary ancestral reconstruction and protein-energy interpretation layer

12.1 Role within the overall framework

The auxiliary layer is deliberately downstream of the primary BABAPPA classifier. Its purpose is to provide mechanistic interpretation for a restricted subset of families, branches, and sites that satisfy conservative branch-level criteria in the main framework. In the default workflow, a family is eligible only when calibrated family-level adaptive probability exceeds a prespecified threshold, at least one branch-level probability exceeds a stricter threshold, artifact probability remains below a conservative ceiling, no abstention or out-of-distribution flag is raised, and phylogenetic information is sufficient for ASR (for example minimum taxon count and branch support).

This design ensures that the auxiliary module does not dilute the central statistical target of the main paper. Instead, it asks a different question: *given a branch-site candidate that is already robust to alignment uncertainty under the main BABAPPA framework, what structural or energetic interpretation is consistent with the reconstructed historical sequence changes?*

12.2 Input objects and uncertainty propagation

For each eligible family g , the auxiliary module consumes the alignment ensemble $\{X_g^{(a)}\}_{a=1}^K$, the chosen tree or tree ensemble, candidate branches from the primary model, site-level saliency or posterior-like importance scores from the primary model, and optional structural templates or structure-prediction metadata.

Because alignment uncertainty affects ASR as well as selection inference, the module must not reconstruct a single ancestor from a single alignment and treat it as ground truth. Instead, ancestral states should be represented as an ensemble:

$$\mathcal{R}_{g,n} = \left\{ R_{g,n}^{(a,m)} : a = 1, \dots, K; m = 1, \dots, M_a \right\},$$

where n indexes an internal node, a indexes alignment workflow, and m indexes posterior samples, bootstrap replicates, or method-specific alternative reconstructions. Alignment-integrated ASR has been shown to improve accuracy relative to single-alignment practice, and recent work suggests that richer generative models can improve ancestor recovery under some conditions [1, 8]. At the same time, ASR appears to be more robust than often assumed in many realistic settings when phylogenetic signal is adequate [25]. The auxiliary design therefore treats uncertainty propagation as a requirement, not an optional refinement.

12.3 Recommended ASR implementation modes

The software should support at least three ASR modes.

Mode A: per-aligner classical ASR. Each alignment is analyzed independently using a classical maximum-likelihood or Bayesian ASR engine. Nodewise posterior probabilities are exported for all residues of interest.

Mode B: alignment-integrated ASR. Reconstructions are combined across alignments with weighting by alignment confidence, posterior probability, or equal weights. The alignment-integrated strategy is strongly motivated by prior work showing improved ancestral accuracy and better downstream structural and functional inference [1].

Mode C: generative or deep ASR. A modern sequence model with explicit generative capacity is used as an alternative or complement to classical ASR, particularly when epistasis or family-specific constraints are expected to matter [8]. This mode should remain optional and clearly versioned because such methods are still comparatively new.

For each internal node n and site j , the output should be a residue distribution rather than only a single MAP call:

$$P(A_{g,n,j} = r \mid \text{data}) \quad \text{for } r \in \mathcal{R},$$

together with an entropy-like uncertainty summary.

12.4 Branchwise substitution-set construction

The main BABAPPA model nominates branch-level candidates and may optionally prioritize a subset of high-importance sites. For a focal branch $b = (u \rightarrow v)$, the auxiliary layer constructs a branchwise substitution set

$$\mathcal{S}_{g,b} = \{(j, r_u, r_v, w_j)\},$$

where j is the site or homologous region identifier, r_u and r_v are ancestor and descendant residues sampled or summarized from the reconstruction ensemble, and w_j is a weight derived from primary-model site support, ASR confidence, or both. In practice, the set may be restricted to sites with high primary-model attribution, low posterior ambiguity under ASR, and placement within domains or structural regions of interest.

This substitution set defines the mechanistic object that will be evaluated downstream. Importantly, it is not interpreted as proof of adaptation. It is interpreted as the collection of historical amino-acid changes most plausibly associated with a robust branch-site event.

12.5 Protein structure and energy proxies

The auxiliary module should support a pluggable structural-analysis backend. Because the exact engine may evolve during project execution, this manuscript leaves the implementation-specific program name as a placeholder, but requires for each reconstructed sequence or sampled ancestor a predicted three-dimensional structure or structure ensemble, residue-level confidence metrics, a global stability-like score, residewise or regionwise energetic frustration or strain proxies, interface or binding-surface scores where applicable, and optional conformational or flexibility proxies.

For each focal branch, the module should compare ancestor and descendant ensembles and estimate a distribution of energetic changes:

$$\Delta E_{g,b}^{(s)} = E(\text{descendant}^{(s)}) - E(\text{ancestor}^{(s)}),$$

or analogous quantities for any chosen score E . The goal is not to recover an absolute physical

free energy with experimental precision, but to obtain a coherent comparative proxy for whether the nominated substitutions are associated with stability-like, interface-like, or conformationally informative shifts. Energy-landscape thinking is increasingly recognized as a useful way to interpret ASR-based evolutionary studies, particularly when substitutions alter conformational access or functional constraints rather than simply increasing or decreasing stability [6].

12.6 Outputs of the auxiliary layer

For each eligible family and branch, the auxiliary module should produce an ASR uncertainty report for focal nodes, a weighted list of candidate historical substitutions, branchwise distributions of stability-like or energy-like changes, sitewise structural annotations mapping candidate substitutions onto domains/interfaces/cores/surfaces or predicted flexible regions, and an interpretation summary (for example predominantly stability-like, predominantly interface-like, conformationally suggestive, or mechanistically unresolved).

The final report should clearly label all such summaries as *auxiliary mechanistic interpretation*. They are not to be combined with the main adaptive probability into one opaque composite score unless a later study explicitly validates such integration.

12.7 Evaluation of the auxiliary layer

The auxiliary module should be evaluated separately from the primary selection detector, with targets including ASR accuracy on simulated ancestors, robustness of ASR summaries across alignment workflows, rank correlation between true simulated energetic perturbations and predicted branchwise energy proxies when latent quantities are available, localization accuracy for structurally causal substitutions in simulation studies with explicit energy- or structure-linked fitness effects, and empirical case studies anchored in literature-supported mechanistic hypotheses.

In other words, the auxiliary layer is validated as a *mechanistic interpretation module*, not as the principal branch-site selection statistic.

13 Planned empirical workflow

Although the present article does not yet include executed empirical results, the intended empirical workflow is straightforward. It begins with curated orthogroups that have verified coding sequences, followed by alignment-ensemble generation for each orthogroup. Next, an ML tree is inferred for each workflow or a trusted species-tree mapping protocol is applied. Before inference, empirical saturation diagnostics are computed using `babappa_saturation.py` or the integrated saturation module.

The intended mature BABAPPA run will produce calibrated branch- and site-level probabilities, artifact probabilities, saturation-risk scores, and conservative gene-level support derived from site evidence. In the current executable implementation, the validated route is site-level scoring followed by site-to-gene aggregation; direct gene-level classifiers are retained mainly as diagnostic baselines. Classical and uncertainty-aware baseline methods should be run on the

same orthogroups for direct comparison. Cross-method concordance should be evaluated, with special attention to cases that appear selection-like under classical methods but are classified as artifact-like or low-identifiability by BABAPPA.

The auxiliary ASR/energy layer is triggered only for robust, cross-aligner-stable, low-artifact candidates that pass branch-level confidence criteria. Focal ancestors are reconstructed under alignment-aware ensembles with propagated uncertainty, and branchwise structural or energy-like shifts are summarized. Final reports explicitly separate primary selection inference from auxiliary mechanistic interpretation.

The empirical panel, accession information, preprocessing details, hardware, and frozen-model identifier should be filled later using the placeholders [PLACEHOLDER_EMPIRICAL_PANEL], [PLACEHOLDER_HARDWARE], and [PLACEHOLDER_ZENODO_DOI].

14 Placeholder results and reporting template

To keep this manuscript implementation-ready, I recommend that downstream execution fill the following structured placeholders rather than rewriting sections ad hoc. The subsection below records the current engineering status so that it is not confused with final biological results.

Current engineering benchmark status

The current code base has passed end-to-end validation through simulation, saturation-panel construction, alignment scaffolding, external-alignment method policy, tensorization, dataset indexing, oracle site-label extraction, site-dataset construction, leakage auditing, site baseline training, site neural training, calibration, threshold-policy profiling, stratified evaluation, site-to-gene aggregation, aggregation null controls, repeated-seed stability benchmarking, report generation, cross-tier summarization, and run-summary validation. The most important scientific correction made during development was the shift away from direct whole-gene binary classification.

The direct gene-level phase was informative but insufficient. A first 1000-family moderate-saturation neural run collapsed to essentially all-negative predictions at threshold 0.5 and all-split AUROC approximately 0.5165. A later gene-level neural-v2 model improved the moderate-saturation 1000-family setting, reaching all-split AUROC approximately 0.7886 and test AUROC approximately 0.7390, but four-tier saturation tests showed unstable ranking and probability compression. Saturation-aware embeddings, group-weighted losses, balanced samplers, ranking losses, and site-attention variants did not make direct gene-level classification robust. Label-signal auditing then showed that the biologically meaningful signal was sparse and site-localized.

The implemented site-level formulation resolves this dilution problem in the current simulation setting. In the completed 10,000-family four-tier benchmark, oracle selected-site labels were extracted from simulator truth, leakage-prone truth metadata were excluded from input features, and a downsampled site dataset with 1,645,380 rows was used for site-level learning. The site neural model achieved all-split AUROC 0.9817 and test AUROC 0.9831. Tier-specific AUROC values remained high but declined gracefully with saturation pressure: low 0.9934, moderate

0.9878, high 0.9749, and extreme 0.9515. Repeated-seed stability was strong, with mean test AUROC approximately 0.9789 and standard deviation approximately 0.0019. Aggregating site probabilities by maximum site probability recovered gene-level simulator labels with AUROC 1.0 and F1 1.0 across split, method, and saturation tiers. Null and decoy aggregation controls remained near random, with control AUROC means approximately 0.50–0.54 and empirical $p = 0.0196$.

The fast external-aligner 10,000-family validation is also complete across low, moderate, high, and extreme saturation tiers. Each tier contains 2,500 simulated families and uses the production-fast ensemble `identity`, `MAFFT`, `BABAPPAlign`, and `MUSCLE` with automatic method quarantine. Low-tier `BABAPPAlign` was quarantined because 38 of 2,500 family-method attempts failed, giving a failure fraction of 0.0152 above the 0.01 policy threshold; `identity`, `MAFFT`, and `MUSCLE` remained usable. In moderate, high, and extreme tiers, all four production-fast methods passed method policy. Site-neural test AUROC values were 0.9945, 0.9835, 0.9726, and 0.9417 for low, moderate, high, and extreme tiers, respectively. Site-to-gene test AUROC values were 1.0, 0.9999, 1.0, and 1.0. Aggregation controls remained near-random, with shuffled-label control means approximately 0.50 and random or permutation controls approximately 0.53, while observed aggregation AUROC remained approximately 0.997–1.0.

The conservative explicit branch-truth 100,000-family Apple Silicon/MPS validation is now also complete across low, moderate, high, and extreme saturation tiers. Each tier contains 25,000 simulated families with 8 taxa and 300 codons, uses the production-fast ensemble `identity`, `MAFFT`, `BABAPPAlign`, and `MUSCLE`, and applies the `conservative_branch_site` feature policy. The plan completed 104 tier-stage markers with no partial markers. Truth-status audit reported `explicit_truth_ok`, `explicit_simulator_branch_truth`, and `proxy_from_foreground_taxon=Fail` for every tier. Branch-site neural test AUROC values were 0.9986, 0.9970, 0.9945, and 0.9900 for low, moderate, high, and extreme tiers, respectively. Branch-level aggregation all-split AUROC values were 0.9998, 0.9994, 0.9988, and 0.9979. Branch-to-gene aggregation all-split AUROC values were 0.9997, 0.9996, 0.9995, and 0.9994. Global shuffled-branch-label controls remained near random at approximately 0.499 AUROC, whereas some prevalence-preserving or family-preserving controls remained high and should be interpreted as partial rather than fully destructive controls.

These results should be interpreted as simulation-oracle branch-site validation, not as final empirical branch-site BABAPPA performance. The target $y_{\text{branch,site}}$ is allowed as a supervised label in simulation, but it is never allowed as an input feature. The branch and branch-to-gene aggregation results show that sparse branch-site evidence can recover higher-level simulator support under the current simulation design. The completed external-aligner validations support the feasibility of the production-fast method set `identity`, `MAFFT`, `BABAPPAlign`, and `MUSCLE` with quarantine. PRANK should remain diagnostic-only, and T-Coffee should remain optional diagnostic-only. The warning `branch_context_ablation:context_only_shortcut_high` remains important and motivates conservative feature policy, ablation reporting, and cautious empirical deployment.

Training corpus

Figure placeholder: End-to-end BABAPPA workflow.
Suggested panel layout for final manuscript: (A) simulation and truth extraction, (B) alignment and aligned-site map construction, (C) oracle site-label extraction and leakage-audited site dataset construction, (D) site baseline and neural training, (E) calibration, threshold-policy selection, and site-to-gene aggregation, (F) conservative empirical deployment and optional auxiliary ASR/energy interpretation.

Figure 1: Planned graphical overview of the BABAPPA computational workflow and decision layers.

Figure placeholder: 100,000-family explicit branch-truth Apple Silicon/MPS saturation profile.
Suggested final content: branch-site neural AUROC, branch-level aggregation AUROC, branch-to-gene aggregation AUROC, destructive control AUROC, truth-status audit, and context-ablation warning status reported separately for low/moderate/high/extreme saturation tiers. Explicit branch-site neural test AUROC values are low 0.9986, moderate 0.9970, high 0.9945, and extreme 0.9900. Branch-level all-split AUROC values are low 0.9998, moderate 0.9994, high 0.9988, and extreme 0.9979. Branch-to-gene all-split AUROC values are low 0.9997, moderate 0.9996, high 0.9995, and extreme 0.9994.

Figure 2: Saturation-stratified explicit branch-truth 100,000-family Apple Silicon/MPS performance summary for manuscript-ready reporting.

Training-corpus field	Value
Total simulated families	100,000 in the completed conservative explicit branch-truth Apple Silicon/MPS validation; earlier completed development benchmarks include 10,000 controlled site-level families and 10,000 fast external-aligner families
Train/validation/calibration/test counts	350,000 branch-site neural rows per tier after streamed/capped construction; branch-level aggregation all-split rows approximately 173,654 per tier; branch-to-gene aggregation all-split rows approximately 63,099 per tier
Taxon-count range	8 taxa in the completed 100,000-family explicit branch-truth validation; 16 taxa in the earlier completed 10,000-family site-level benchmark
Mean sequence length	300 codons in the completed 100,000-family explicit branch-truth validation
Mean indel burden	[PLACEHOLDER]
Mean recombination breakpoints	[PLACEHOLDER]
Saturation-tier counts (low/moderate/high/extreme)	25,000 families per tier in the completed 100,000-family explicit branch-truth validation; production-fast methods were identity, MAFFT, BABAPPA1ign, and MUSCLE

Site-level dataset size	1,645,380 downsampled site rows with 149,580 positive sites in the earlier controlled 10,000-family benchmark; 350,000 branch-site neural rows per tier in the completed 100,000-family explicit branch-truth validation
-------------------------	--

Training procedure

Training field	Value
Optimizer, scheduler, batch size	AdamW site/branch-site neural training; batch size 256 in the completed 10,000-family run and batch size 64 in the completed 100,000-family Apple Silicon/MPS branch-site run
Epochs and early-stopping rule	30-epoch user-run template with validation-AUROC monitoring; best checkpoint selected by validation metric
Hardware and runtime	Completed 100,000-family explicit branch-truth validation on Apple Silicon/MPS with 36 GB unified memory using streamed/capped outputs and conservative memory safeguards
Best checkpoint identifier	<code>site_neural_saturation_10000</code> ; 100,000-family branch-site neural outputs are retained in the corresponding <code>branch_site_neural_explicit_branch_truth_100k_mps_*</code> directories

Benchmarking

Benchmark output	Value
Gene-level AUROC/AUPRC versus base-lines	Direct gene-level models are retained as diagnostics; site-to-gene and branch-to-gene aggregation are the supported routes. In the 100,000-family explicit branch-truth validation, branch-to-gene all-split AUROC was low 0.9997, moderate 0.9996, high 0.9995, and extreme 0.9994.
Branch-level AUROC/AUPRC versus base-lines	In the 100,000-family explicit branch-truth validation, branch-level all-split AUROC was low 0.9998, moderate 0.9994, high 0.9988, and extreme 0.9979.

Site-level localization metrics on synthetic truth	Earlier controlled site neural all AUROC 0.9817 and test AUROC 0.9831; 100,000-family explicit branch-site neural test AUROC low 0.9986, moderate 0.9970, high 0.9945, extreme 0.9900.
False-positive control on hard negatives	Earlier aggregation null/decoy controls near random: mean AUROC approximately 0.50–0.54; empirical $p = 0.0196$. In the 100,000-family explicit branch-truth run, global shuffled-branch-label controls were near random at approximately 0.499 AUROC, while prevalence-preserving controls remained partial rather than fully destructive.
Held-out-aligner or external-aligner performance	Fast external-aligner 10K validation complete: site-neural test AUROC low 0.9945, moderate 0.9835, high 0.9726, extreme 0.9417; site-to-gene test AUROC low 1.0, moderate 0.9999, high 1.0, extreme 1.0. The 100,000-family branch-site run used the same production-fast method set: <code>identity</code> , <code>MAFFT</code> , <code>BABAPPAlign</code> , and <code>MUSCLE</code> .
Stress-test curves (recombination, SRV, MNMs, saturation)	[PLACEHOLDER]
Saturation-tier performance and abstention tables	Controlled site AUROC: low 0.9934, moderate 0.9878, high 0.9749, extreme 0.9515. Fast external-aligner 10K site AUROC: low 0.9945, moderate 0.9835, high 0.9726, extreme 0.9417. Explicit branch-truth 100K branch-site neural test AUROC: low 0.9986, moderate 0.9970, high 0.9945, extreme 0.9900.
Threshold-policy profile (strict-FDR, max-F1, max-MCC, balanced-Youden, high-precision, high-recall)	Site-level strict FDR remains difficult; quantile calibration improves ECE; aggregation-level thresholds are preferred for gene-level discovery mode.

Empirical deployment

Empirical output field	Value
------------------------	-------

Orthogroups analyzed	Two WRKY-family diagnostic pilots have been processed in the current empirical bridge. <code>WRKY_candidate_01</code> was retained as an out-of-domain stress test; <code>WRKY_candidate_02_close</code> was the first in-domain biological diagnostic pilot.
Robust branch-level hits	<code>WRKY_candidate_02_close</code> produced a BABAPPA diagnostic-positive result with maximum gene support 0.177189 and 6,954 called branch-site rows. This is not a discovery claim.
Selection-like cases reclassified as artifact-like or out-of-domain	<code>WRKY_candidate_01</code> had mean pairwise p-distance 0.725799, was classified as out-of-domain/diagnostic-only, and produced no interpretable positive call.
Applicability gate	<code>WRKY_candidate_02_close</code> had 7 taxa, aligned mean pairwise p-distance approximately 0.101293, prefilter decision <code>accept</code> , applicability <code>in_domain</code> , selected tier <code>moderate</code> , and scoring device <code>mps</code> .
Reference and calibration status	codeml/HyPhy-style references and simulation-matched null calibration are required before biological interpretation. The current status is diagnostic-positive pending reference/calibration interpretation.
Families entering auxiliary ASR/energy analysis	None in the present release; ASR/energy interpretation remains downstream and optional.
Representative biological examples	<code>WRKY_candidate_02_close</code> is a diagnostic empirical pilot only, not evidence of discovered positive selection.

Empirical claim boundary

The empirical bridge deliberately separates software success from biological discovery. No simulator truth is available or used during empirical inference. `WRKY_candidate_01` demonstrated that BABAPPA can abstain from a saturated, divergent plant WRKY panel: the family had mean p-distance 0.725799 and was treated as an out-of-domain failure-mode probe. `WRKY_candidate_02_close` demonstrated that the same pipeline can process a closer Brassicaceae-like WRKY panel end to end: 7 taxa, approximately 490 codons, aligned mean p-distance near 0.101293, accepted OOD prefilter, in-domain applicability, moderate-tier model selection, MPS scoring, diagnostic-positive class, maximum gene support 0.177189, and 6,954 called branch-site rows. These outputs are guarded simulation-trained diagnostic evidence. They are not manuscript-ready empirical positive-selection claims until codeml/HyPhy-style reference com-

parison, matched-null calibration, and close-taxa negative controls have been completed and interpreted.

15 Discussion

The central claim of BABAPPA is not that deep learning replaces codon models by fiat, nor that simulator-trained classifiers can recover all historical adaptive events. Rather, the claim is that robust branch-site inference requires explicit exposure to the same nuisance processes that undermine conventional analyses, together with a training objective that rewards alignment-invariant signal and penalizes aligner-specific artifact. This is a narrower claim than “MSA-free selection detection,” but it is a more defensible and immediately actionable one.

Several aspects of the framework deserve emphasis. Using multiple alignments for the same simulated family is a methodological necessity, not a cosmetic ensemble trick, because it forces direct confrontation with homology uncertainty. Explicit latent-homology tracking also gives the supervised objective a rigorous definition that empirical data cannot provide directly. Hard-negative artifact families are as important as positive families, because they teach the model when selection-like patterns should be rejected.

Calibration and abstention are equally central. Researchers need to know not only when the model predicts selection, but also when the data are too confounded or too far outside the training domain for a reliable claim. This point is especially important for saturation. A trustworthy method should explicitly indicate when deep multiple-hit accumulation has destroyed branch-site identifiability, rather than returning an overconfident score.

I also argue that incorporating the second direction as an auxiliary layer is methodologically superior to presenting it as an independent primary selection framework. The recent ASR and protein-energy literature makes that distinction important. Alignment-aware ASR can improve downstream inference, energy-landscape analysis can illuminate how evolutionary changes reshape stability or conformational preferences, and newer generative ASR strategies may capture epistatic structure more effectively [1, 6, 8]. However, none of these developments implies that structural or energetic change alone is a statistically sufficient test of positive selection. By keeping ASR and energy-landscape analysis explicitly downstream of the primary branch-site detector, BABAPPA preserves a clean separation between *evidence that a robust adaptive-like signal is present* and *hypotheses about what that signal may have done mechanistically*. In my view, that separation makes the paper stronger and more reviewer-resistant.

The framework also has limitations. Any simulator, no matter how rich, remains a stylized approximation to reality. Overfitting to simulator peculiarities is therefore a major risk, and held-out nuisance regimes, held-out aligners, and empirical stress testing are essential. Site-level localization on empirical data remains intrinsically harder than gene-level or branch-level prediction because latent homology cannot be observed directly. Moreover, a model that is robust to alignment uncertainty is not automatically robust to errors in orthogroup construction, hidden paralogy, or annotation mistakes; these should be treated as additional future nuisance classes. The auxiliary module introduces its own limitations as well: ASR uncertainty, structure-prediction uncertainty, and imperfect correspondence between energetic proxies and organismal fitness all mean that mechanistic summaries must remain explicitly probabilistic

and interpretive.

The current software status adds a practical limitation to this conceptual list. The implemented neural model is now site-level and branch-site-level rather than direct gene-level, and the completed controlled, fast external-aligner, and conservative explicit branch-truth benchmarks support the claim that the oracle-supervised formulation is learnable and stable under four-tier saturation pressure. However, this remains simulation-supervised method development. The target labels are derived from simulator truth, and the current branch-to-gene aggregation result is an upper-bound validation of the information pathway rather than an empirical proof of selection inference. The external-aligner result validates a feasible production-fast ensemble of `identity`, `MAFFT`, `BABAPPAlign`, and `MUSCLE` with automatic quarantine. `PRANK` and `T-Coffee` should remain diagnostic-only unless explicitly used for stress testing. The empirical bridge, pilot-panel tooling, OOD gate, and deployable model package are now implemented, but broad real-world performance, calibrated empirical p-like values, and concordance with `codeml`/`HyPhy`-style reference workflows remain unproven. The context-only shortcut warning observed in branch-context ablation should remain a visible caveat whenever branch-conditioned results are reported.

Despite these limits, `BABAPPA` provides a coherent path toward a serious new class of methods in molecular evolution. Instead of relying on one alignment and one codon model, it operationalizes a stricter principle: infer adaptive signal only after confronting alternative homology hypotheses and major confounders in a supervised, calibrated, and benchmarked framework. The auxiliary ASR/energy layer then extends the workflow from detection to interpretation without collapsing distinct inferential levels into a single score.

The present conclusion is therefore bounded. `BABAPPA` is now a validated research-alpha framework with explicit simulator branch-site truth validation at 100,000 families, a deployable simulation-trained conservative model package, and a guarded empirical diagnostic workflow. It has proven scalable explicit-truth simulation validation, conservative feature-policy robustness, feasible production alignment ensembles, model packaging, and empirical OOD abstention. It has not proven final empirical positive-selection inference, broad real-world performance, `codeml`/`HyPhy` concordance, or calibrated empirical p-values. Future manuscript claims should require simulation-matched calibration, close-taxa negative controls, and externally interpretable reference comparisons before any empirical discovery statement is made.

Author contributions

Krishnendu Sinha conceived the project, designed the framework, wrote the manuscript, and is solely responsible for the conceptual, methodological, and computational specification presented here.

Competing interests

The author declares no competing interests.

Data and code availability

At the current stage, the code repository, frozen model artifacts, simulation manifests, and benchmark outputs will be deposited at [PLACEHOLDER_REPOSITORY_URL] and archived at [PLACEHOLDER_ZENODO_DOI]. These entries are placeholders and should be replaced upon project execution and release. The manuscript currently refers to software release 0.5.1-alpha; the exact commit used for the final manuscript should be recorded as [PLACEHOLDER_COMMIT_HASH]. Reproducible report outputs should include `report_summary.json`, `run_summary.json`, leakage-audit outputs, site-dataset manifests, site-neural and branch-site-neural metrics, calibration and threshold-policy outputs, site-to-gene and branch-to-gene aggregation outputs, aggregation-control outputs, stability-benchmark summaries, external-aligner site-map reports, method-policy reports, truth-status audits, cross-tier summary artifacts, and saturation-panel or merged-dataset manifests where applicable. Large raw branch-site label TSVs and other streamed intermediates may be pruned after downstream summaries, audits, logs, and model artifacts have been preserved; such pruning should be recorded in cleanup manifests rather than treated as loss of the reproducible scientific conclusion.

References

- [1] Kelsey Aadland and Bryan Kolaczkowski. Alignment-integrated reconstruction of ancestral sequences improves accuracy. *Genome Biology and Evolution*, 12(9):1549–1565, 2020.
- [2] Sandra Álvarez-Carretero and Ziheng Yang. Beginner’s guide on the use of PAML to detect positive selection. *Molecular Biology and Evolution*, 40(4):msad041, 2023.
- [3] Maria Anisimova, Rasmus Nielsen, and Ziheng Yang. Effect of recombination on the accuracy of the likelihood method for detecting positive selection at amino acid sites. *Genetics*, 164(3):1229–1236, 2003.
- [4] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- [5] P. Bryn Chi and David A. Liberles. Detecting signatures of positive selection against a backdrop of multiple nonadaptive processes. *Molecular Biology and Evolution*, 37(11):3353–3367, 2020.
- [6] Lauren O. Chisholm, Kona N. Orlandi, Sophia R. Phillips, Michael J. Shavlik, and Michael J. Harms. Ancestral reconstruction and the evolution of protein energy landscapes. *Annual Review of Biophysics*, 53:127–146, 2024.
- [7] Ilia I. Davydov, Nicolas Salamin, and Marc Robinson-Rechavi. Large-scale comparative analysis of codon models accounting for protein and nucleotide selection. *Molecular Biology and Evolution*, 36(6):1316–1332, 2019.
- [8] Matteo De Leonardis, Andrea Pagnani, and Pierre Barrat-Charlaix. Reconstruction of ancestral protein sequences using autoregressive generative models. *Molecular Biology and Evolution*, 42(4):msaf070, 2025.

- [9] David A. Duchene, Lindell Bromham, and Sebastien Duchene. Excluding loci with substitution saturation improves phylogenomic inference. *Systematic Biology*, 71(3):676–689, 2022.
- [10] Katherine A. Dunn, Toby Kenney, Hong Gu, and Joseph P. Bielawski. Improved inference of site-specific positive selection under a generalized parametric codon model when there are multinucleotide mutations and multiple nonsynonymous rates. *BMC Evolutionary Biology*, 19(1):22, 2019.
- [11] Robert C. Edgar. MUSCLE: multiple sequence alignment with high accuracy and high throughput. *Nucleic Acids Research*, 32(5):1792–1797, 2004.
- [12] William Fletcher and Ziheng Yang. INDELible: a flexible simulator of biological sequence evolution. *Molecular Biology and Evolution*, 26(8):1879–1888, 2009.
- [13] Walid H. Gharib and Marc Robinson-Rechavi. The branch-site test of positive selection is surprisingly robust but lacks power under synonymous substitution saturation and variation in GC. *Molecular Biology and Evolution*, 30(7):1675–1686, 2013.
- [14] Nick Goldman and Ziheng Yang. A codon-based model of nucleotide substitution for protein-coding dna sequences. *Molecular Biology and Evolution*, 11(5):725–736, 1994.
- [15] Gregory Jordan and Nick Goldman. The effects of alignment error and alignment filtering on the sitewise detection of positive selection. *Molecular Biology and Evolution*, 29(4):1125–1139, 2012.
- [16] Kazutaka Katoh and Daron M. Standley. MAFFT multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- [17] Sergei L. Kosakovsky Pond, Ben Murrell, Mathieu Fourment, Simon D. W. Frost, Wayne Delpont, and Konrad Scheffler. A random effects branch-site model for detecting episodic diversifying selection. *Molecular Biology and Evolution*, 28(11):3033–3043, 2011.
- [18] Sergei L. Kosakovsky Pond, Art F. Y. Poon, Ryan Velazquez, Steven Weaver, Nathan L. Hepler, Ben Murrell, Stephen D. Shank, Brittany R. Magalis, Dave Bouvier, Anton Nekrutenko, Sadie Wisotsky, Stephanie J. Spielman, Simon D. W. Frost, and Spencer V. Muse. HyPhy 2.5—a customizable platform for evolutionary hypothesis testing using phylogenies. *Molecular Biology and Evolution*, 37(1):295–299, 2020.
- [19] Sergei L. Kosakovsky Pond, David Posada, Michael B. Gravenor, Carl H. Woelk, and Simon D. W. Frost. Automated phylogenetic detection of recombination using a genetic algorithm. *Molecular Biology and Evolution*, 23(10):1891–1901, 2006.
- [20] Sergei L. Kosakovsky Pond, David Posada, Michael B. Gravenor, Carl H. Woelk, and Simon D. W. Frost. GARD: a genetic algorithm for recombination detection. *Bioinformatics*, 22(24):3096–3098, 2006.
- [21] Ari Löytynoja. Accurate extension of multiple sequence alignments using a phylogeny-aware graph algorithm. *Bioinformatics*, 28(13):1684–1691, 2012.

- [22] Ari Löytynoja. Phylogeny-aware alignment with PRANK. *Methods in Molecular Biology*, 1079:155–170, 2014.
- [23] Anthony G. Lucaci, Sadie R. Wisotsky, Stephen D. Shank, Spencer V. Muse, and Sergei L. Kosakovsky Pond. Evolutionary shortcuts via multi-nucleotide substitutions and their impact on natural selection analyses. *Molecular Biology and Evolution*, 40(7):msad150, 2023.
- [24] Bui Quang Minh, Heiko A. Schmidt, Olga Chernomor, Dominik Schrempf, Michael D. Woodhams, Arndt von Haeseler, and Robert Lanfear. IQ-TREE 2: new models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37(5):1530–1534, 2020.
- [25] Ricardo Muñoz-Trejo, Yeonwoo Park, and Joseph W. Thornton. Robustness of ancestral sequence reconstruction to among-site and among-lineage evolutionary heterogeneity. *Molecular Biology and Evolution*, 42(4):msaf084, 2025.
- [26] Ben Murrell, Steven Weaver, Martin D. Smith, Joel O. Wertheim, Sasha Murrell, Amy Aylward, Kadri Eren, Thomas Pollner, Daniel P. Martin, David M. Smith, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Gene-wide identification of episodic selection. *Molecular Biology and Evolution*, 32(5):1365–1371, 2015.
- [27] Ben Murrell, Joel O. Wertheim, Sasha Moola, Tim Weighill, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Detecting individual sites subject to episodic diversifying selection. *PLoS Genetics*, 8(7):e1002764, 2012.
- [28] Spencer V. Muse and Brandon S. Gaut. A likelihood approach for comparing synonymous and nonsynonymous nucleotide substitution rates, with application to the chloroplast genome. *Molecular Biology and Evolution*, 11(5):715–724, 1994.
- [29] Cedric Notredame, Desmond G. Higgins, and Jaap Heringa. T-Coffee: a novel method for fast and accurate multiple sequence alignment. *Journal of Molecular Biology*, 302(1):205–217, 2000.
- [30] Benjamin D. Redelings. Erasing errors due to alignment ambiguity when estimating positive selection. *Molecular Biology and Evolution*, 31(8):1979–1993, 2014.
- [31] Benjamin D. Redelings. BAli-Phy version 3: model-based co-estimation of alignment and phylogeny. *Bioinformatics*, 37(17):3032–3034, 2021.
- [32] Konrad Scheffler, Darren P. Martin, and Cathal Seoighe. Robust inference of positive selection from recombining coding sequences. *Bioinformatics*, 22(20):2493–2499, 2006.
- [33] Adam G. Selberg, Sadie Wisotsky, Sergei L. Kosakovsky Pond, and Spencer V. Muse. Minus the error: testing for positive selection in the presence of residual alignment errors. *Molecular Biology and Evolution*, 2025. Published online / early access; final bibliographic details to be updated if needed.
- [34] Martin D. Smith, Joel O. Wertheim, Steven Weaver, Ben Murrell, Konrad Scheffler, and Sergei L. Kosakovsky Pond. Less is more: an adaptive branch-site random effects model

- for efficient detection of episodic diversifying selection. *Molecular Biology and Evolution*, 32(5):1342–1353, 2015.
- [35] Aarti Venkat, Matthew W. Hahn, and Joseph W. Thornton. Multinucleotide mutations cause false inferences of lineage-specific positive selection. *Nature Ecology & Evolution*, 2:1280–1288, 2018.
 - [36] Charlotte West and colleagues. Detecting interspecific positive selection using convolutional neural networks. *Molecular Biology and Evolution*, 42(7):msaf154, 2025.
 - [37] Sadie R. Wisotsky, Sergei L. Kosakovsky Pond, Stephen D. Shank, and Spencer V. Muse. Synonymous site-to-site substitution rate variation dramatically inflates false positive rates of selection analyses: ignore at your own peril. *Molecular Biology and Evolution*, 37(8):2430–2439, 2020.
 - [38] Xuhua Xia, Zheng Xie, Marco Salemi, Ling Chen, and Yong Wang. An index of substitution saturation and its application. *Molecular Phylogenetics and Evolution*, 26(1):1–7, 2003.
 - [39] Ziheng Yang. PAML 4: phylogenetic analysis by maximum likelihood. *Molecular Biology and Evolution*, 24(8):1586–1591, 2007.
 - [40] Jianzhi Zhang, Rasmus Nielsen, and Ziheng Yang. Evaluation of an improved branch-site likelihood method for detecting positive selection at the molecular level. *Molecular Biology and Evolution*, 22(12):2472–2479, 2005.